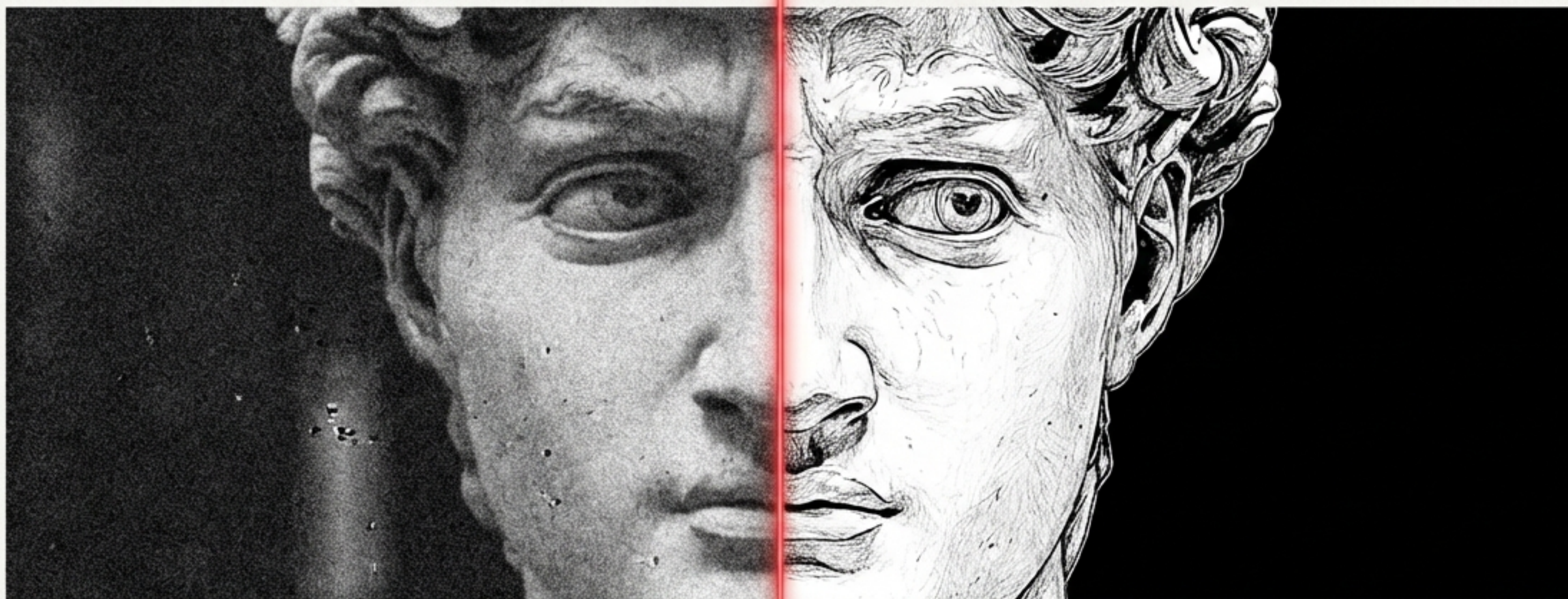


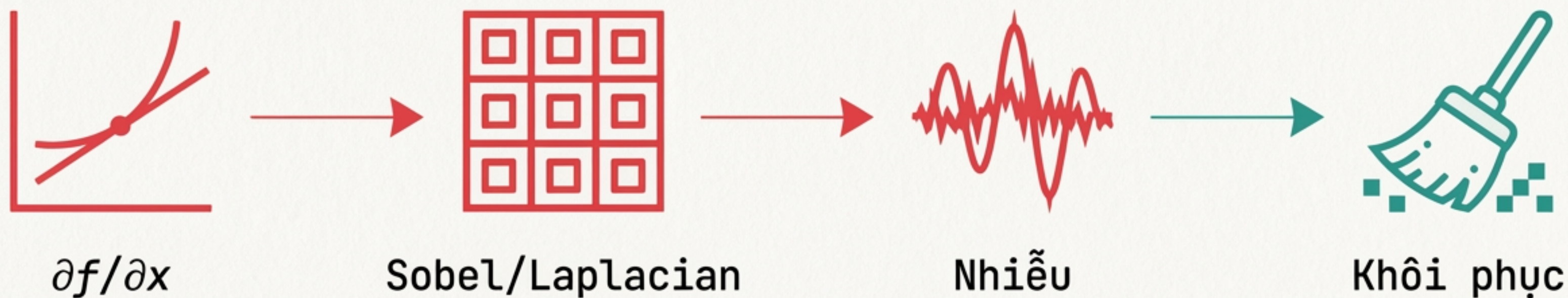
# CHƯƠNG 2: NÂNG CẤP VÀ KHÔI PHỤC ẢNH

## Tiết 5: Phát hiện biên (Edge Detection) & Các bộ lọc Khôi phục



Hành trình từ tín hiệu thô đến cấu trúc hình học.





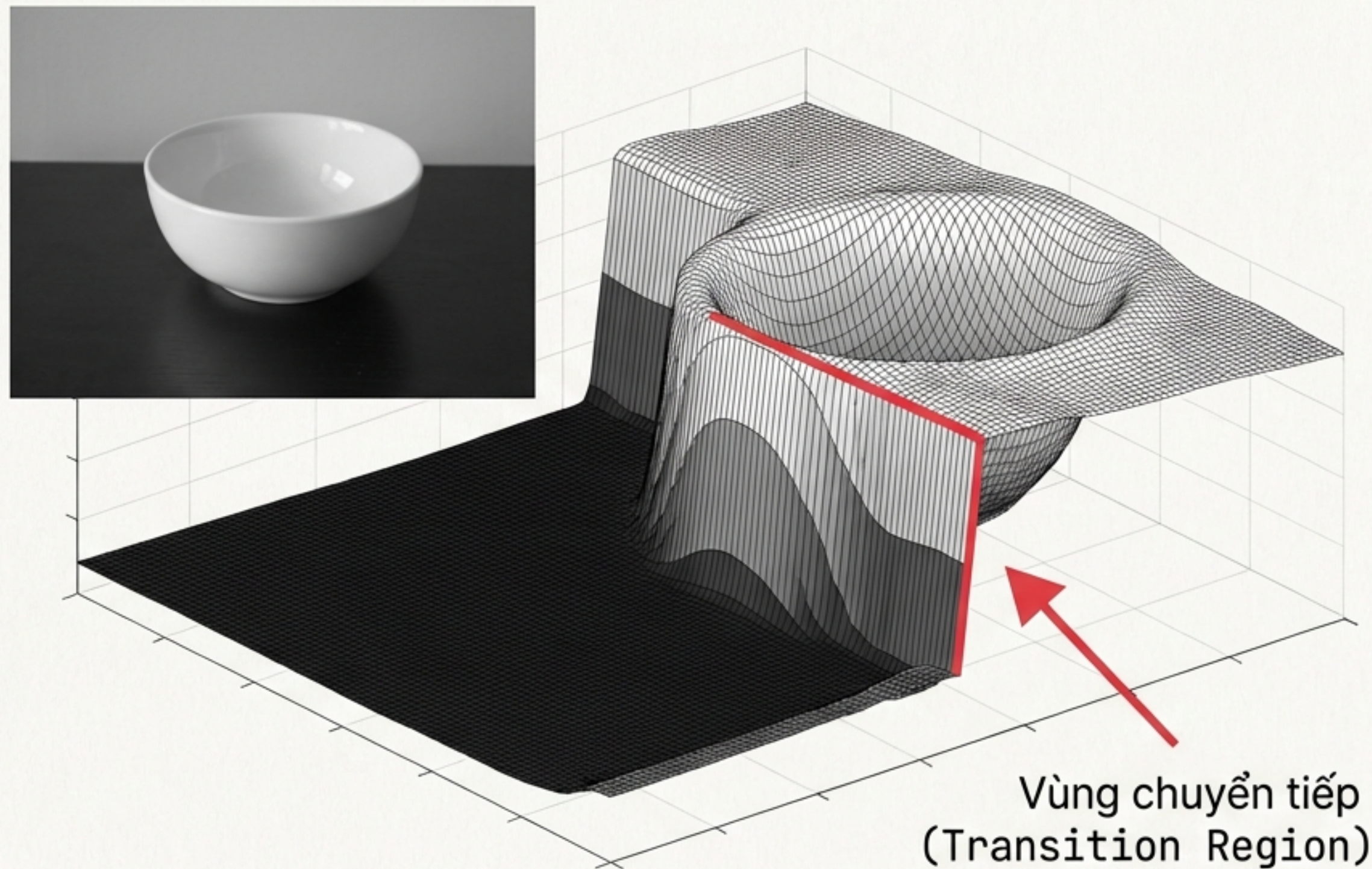
## Helvetica Now Display

- **Hiểu bản chất:** Đường biên là sự thay đổi đột ngột của cường độ sáng.
- **Phân tích công cụ:** Cơ chế hoạt động của Kernel tích chập trong việc tìm biên.
- **Nhận diện vấn đề:** Tại sao không thể đạo hàm trực tiếp trên ảnh nhiễu?
- **Giải pháp thực tiễn:** Ứng dụng lọc Trung vị để xử lý nhiễu muối tiêu (Salt & Pepper).

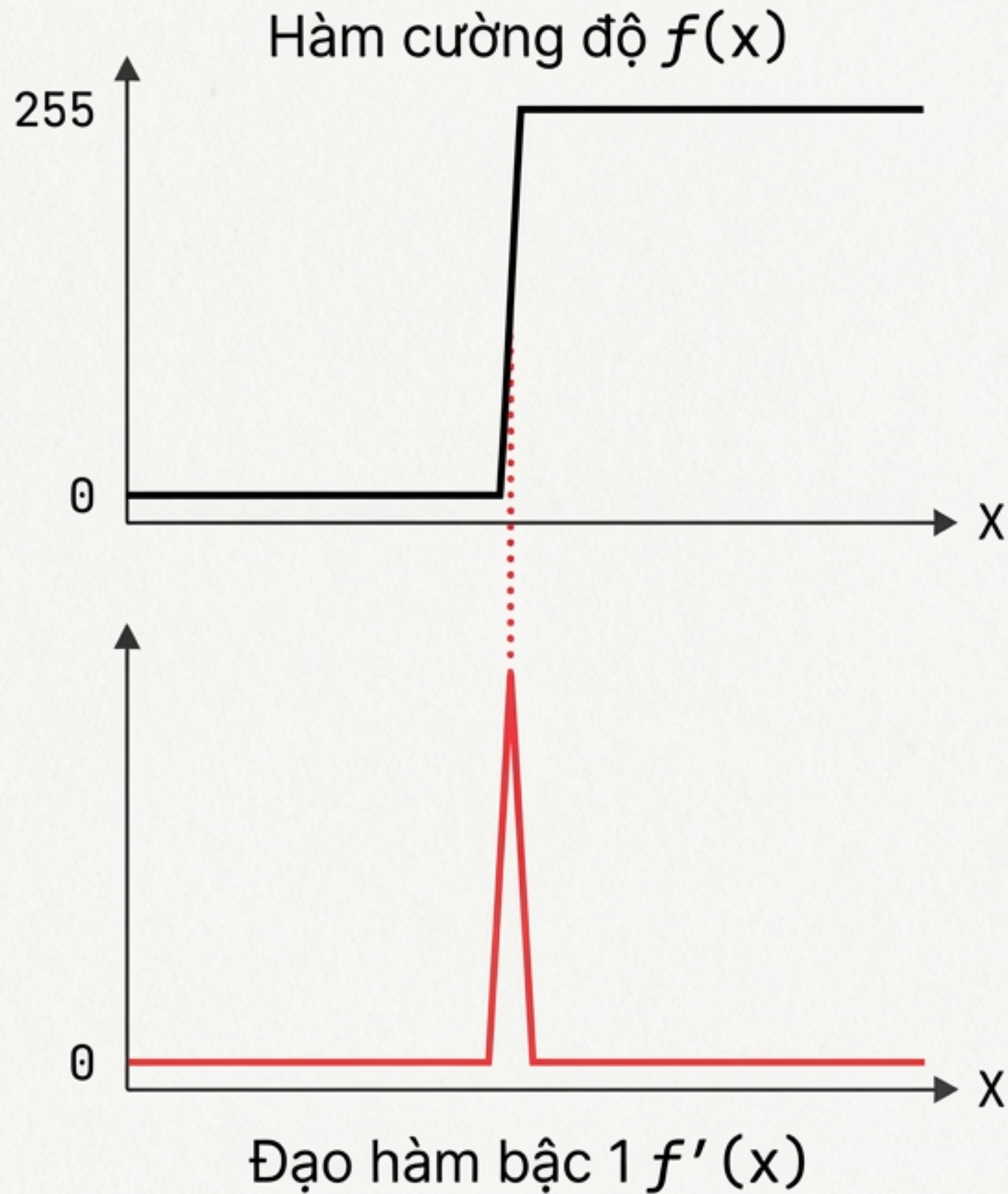


# Bản chất của Đường biên (The Intensity Cliff)

- **Định nghĩa:** Biên là nơi giá trị pixel thay đổi đột ngột nhất.
- **Dữ liệu:** Sự chênh lệch lớn (ví dụ: nhảy từ 0 lên 255).
- **Mô hình:** Bước nhảy (Step Edge).







## Nguyên lý: Đạo hàm đo lường tốc độ thay đổi.

- Quy tắc:
  - Vùng bằng phẳng  $\rightarrow$  Đạo hàm  $\approx 0$ .
  - Đường biên  $\rightarrow$  Đạo hàm đạt Cực trị (Đỉnh/Peak).
- Kết luận: Máy tính tìm biên bằng cách tìm đỉnh của đạo hàm.



# Bộ lọc Sobel - Xấp xỉ Đạo hàm bậc 1

$G_x$  (Sobel X)

|    |   |    |
|----|---|----|
| -1 | 0 | +1 |
| -2 | 0 | +2 |
| -1 | 0 | +1 |

$G_y$  (Sobel Y)

|    |    |    |
|----|----|----|
| -1 | -2 | -1 |
| 0  | 0  | 0  |
| +1 | +2 | +1 |

- Cơ chế: Dùng Tích chập (Convolution).
- Sobel X: Tính đạo hàm phương ngang → Phát hiện biên dọc.
- Sobel Y: Tính đạo hàm phương dọc → Phát hiện biên ngang.



# Tổng hợp Gradient (Gradient Magnitude)



**Chính xác (Pytago):**  $|G| = \sqrt{G_x^2 + G_y^2}$

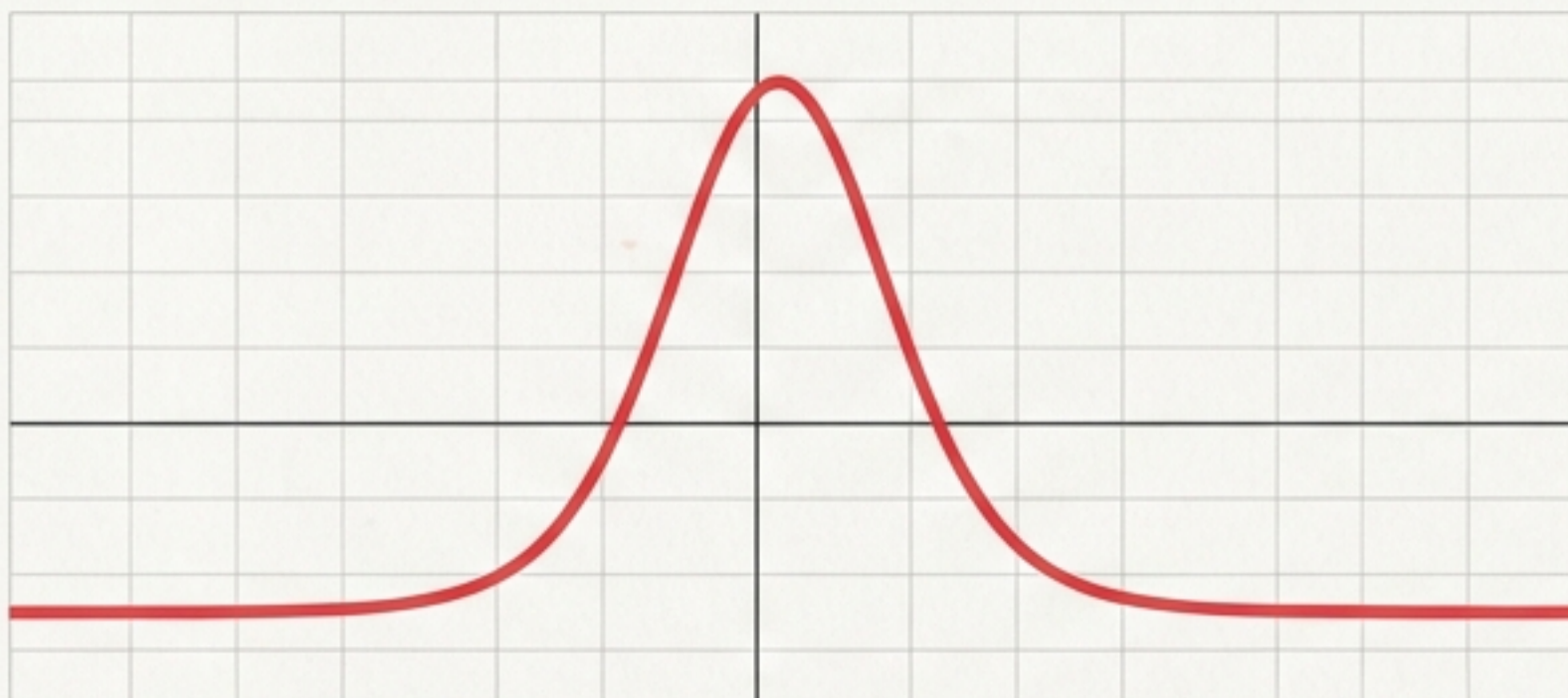
**Xấp xỉ (Hiệu năng):**  $|G| \approx |G_x| + |G_y|$

Một mình Sobel X hay Sobel Y chỉ thấy một nửa sự thật. Độ lớn Gradient cho thấy toàn bộ cấu trúc.

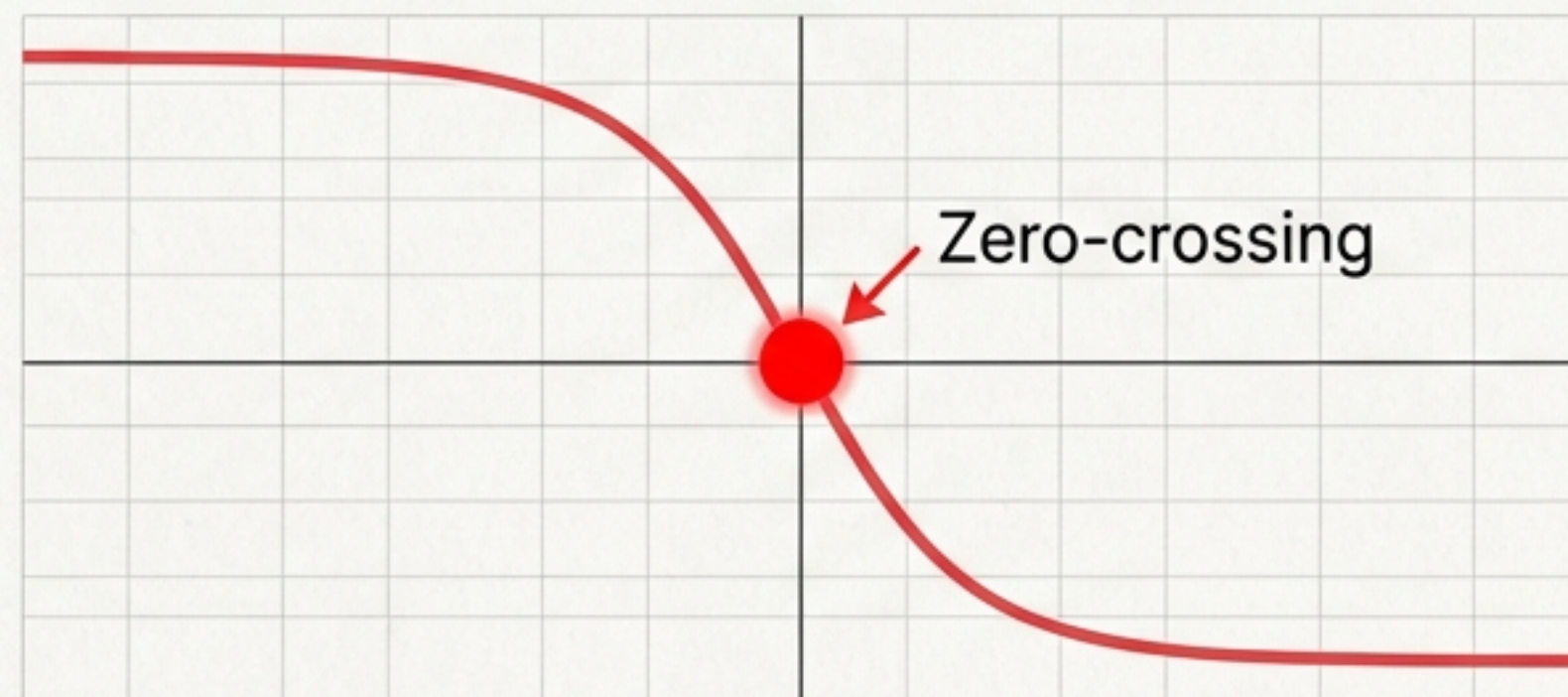


# Đạo hàm bậc 2 - Bộ lọc Laplacian

Bậc 1: Tìm Đỉnh



Bậc 2: Giao điểm không (Zero-crossing)

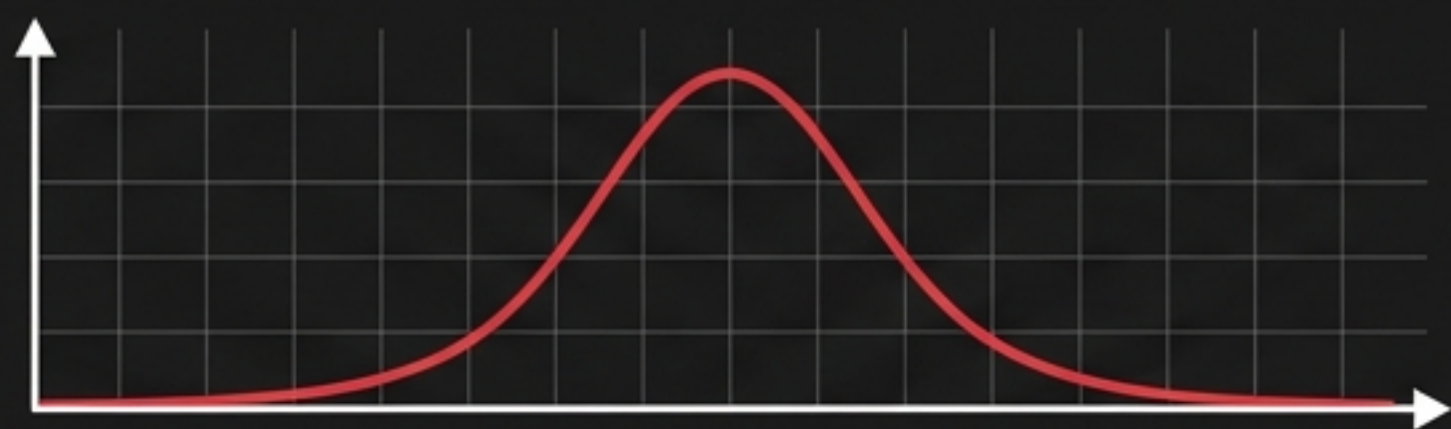
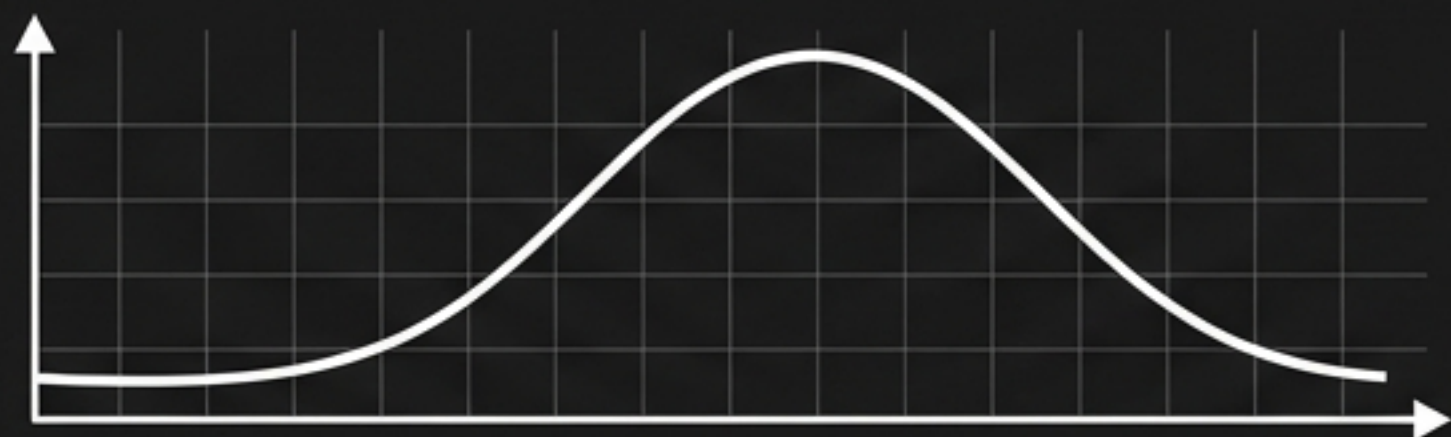


- Cơ chế: Đạo hàm bậc 2 ( $\nabla^2 f$ ).
- Dấu hiệu: Tâm đường biên là nơi đạo hàm đổi dấu.
- Ưu điểm: Đẳng hướng (Isotropic) - Bắt biên mọi hướng chỉ với 1 kernel.

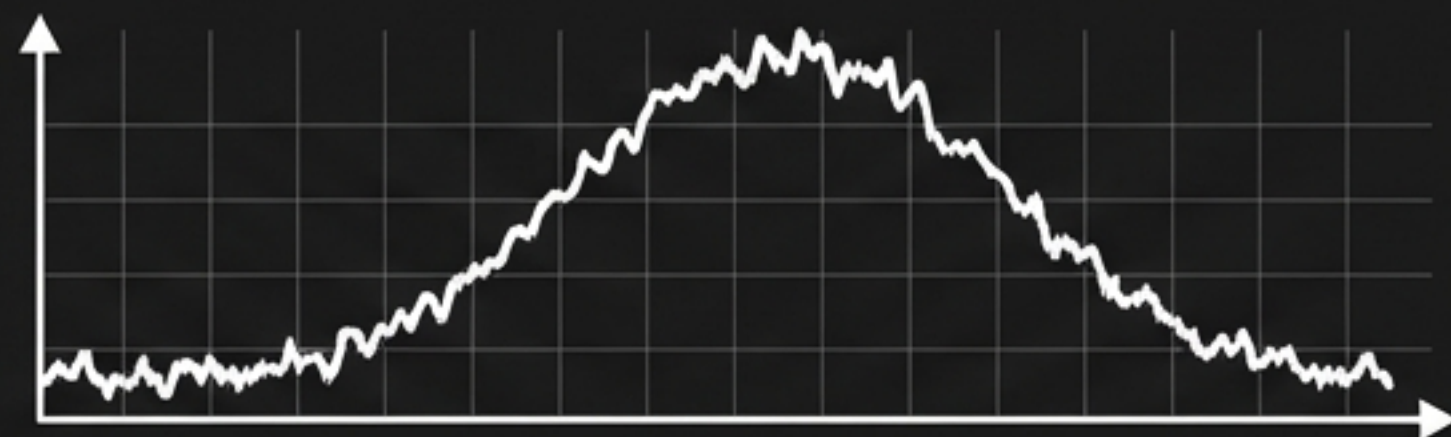
|   |    |   |
|---|----|---|
| 0 | 1  | 0 |
| 1 | -4 | 1 |
| 0 | 1  | 0 |



# Nghịch lý của Đạo hàm & Nhiễu



Tín hiệu sạch



Tín hiệu nhiễu

- **Thực tế:** Ảnh luôn có nhiễu.
- **Hậu quả:** Phép đạo hàm khuếch đại nhiễu cực mạnh.
- **Quy tắc vàng:** Không tìm biên trên ảnh nhiễu mà không làm trơn trước.



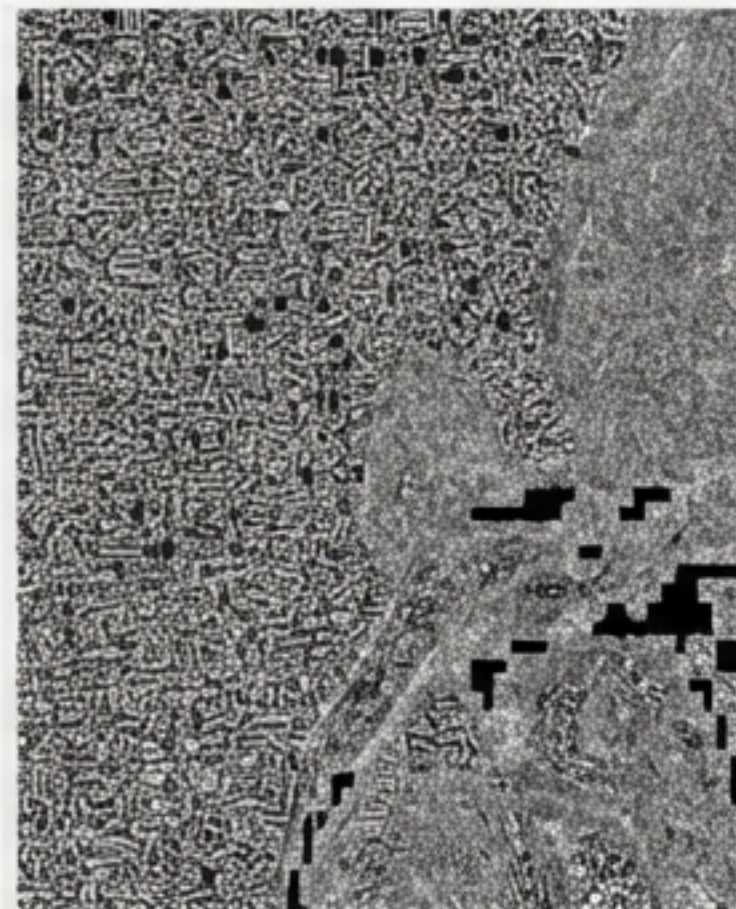
# Chiến lược Khôi phục Ảnh (Image Restoration)

## Lọc Tuyến tính (Mean, Gaussian)

- Cơ chế: Trung bình cộng.
- Nhược điểm: Làm mờ biên, không trị được nhiễu muối tiêu.

## Lọc Phi tuyến (Median, Min, Max)

- Cơ chế: Thống kê thứ tự.
- Ưu điểm: Bảo toàn cạnh sắc nét, loại bỏ nhiễu đột biến.



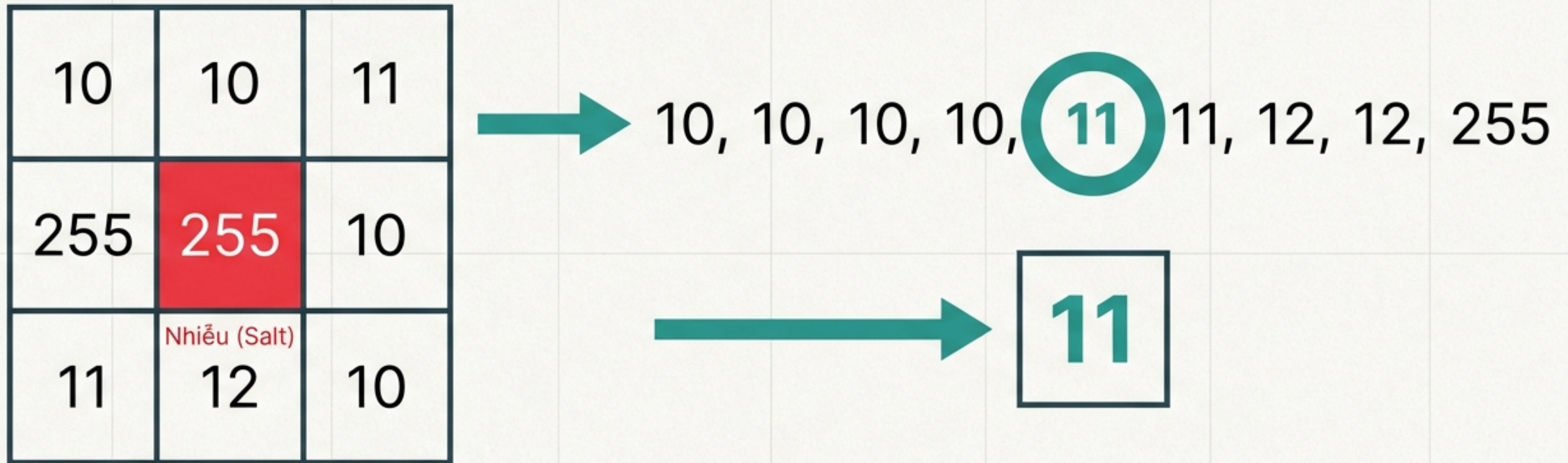
Ảnh Nhiễu (Noisy)



Ảnh Khôi phục (Restored)



# Lọc Trung vị (Median Filter) - Khắc tinh nhiễu muối tiêu



- **Cách làm:** Sắp xếp giá trị trong cửa sổ, chọn giá trị giữa.
- **Hiệu quả:** Giá trị nhiễu (255) bị đẩy về cuối hàng và bị loại bỏ.



# So sánh: Mean vs. Median



Ảnh gốc (Original)



Mean Filter (Thất bại)



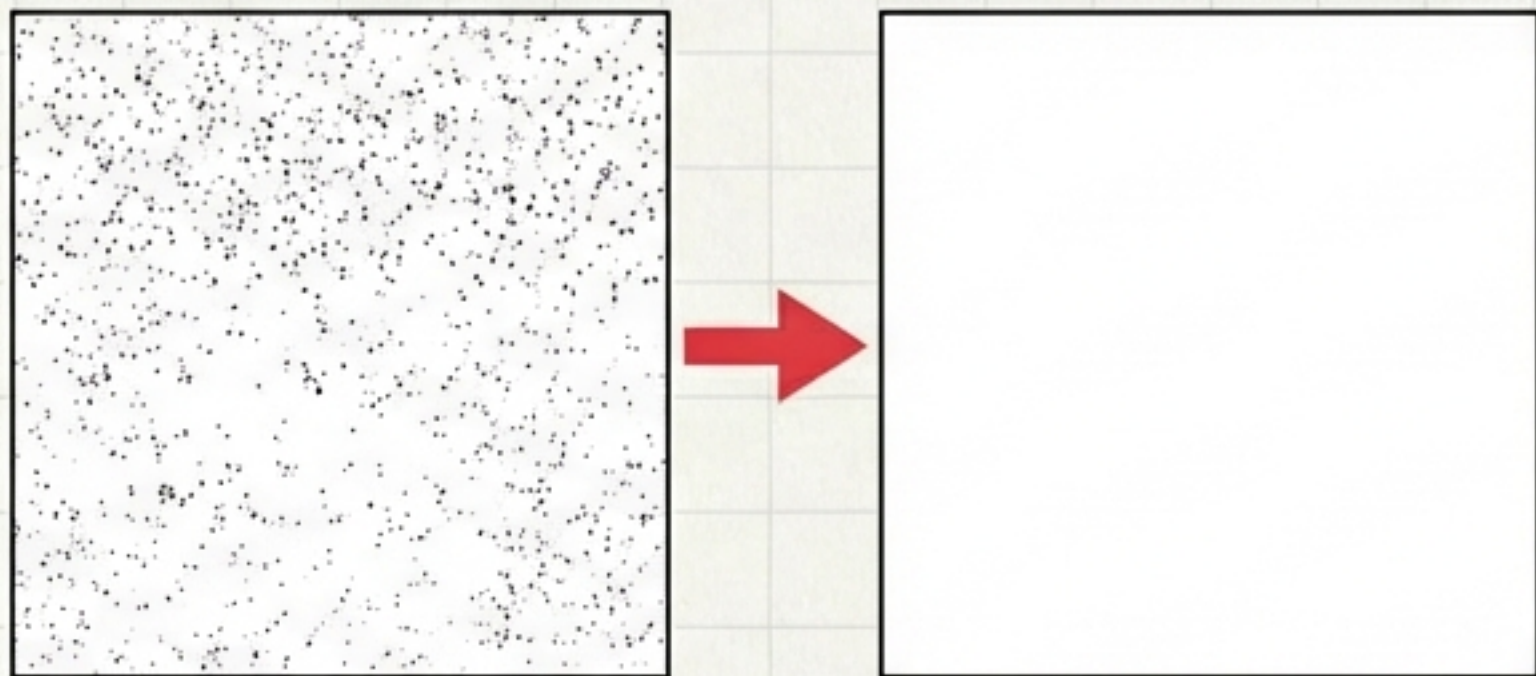
Median Filter (Thành công)

Mean nhạy cảm với ngoại lai. Median kháng ngoại lai (Robust).



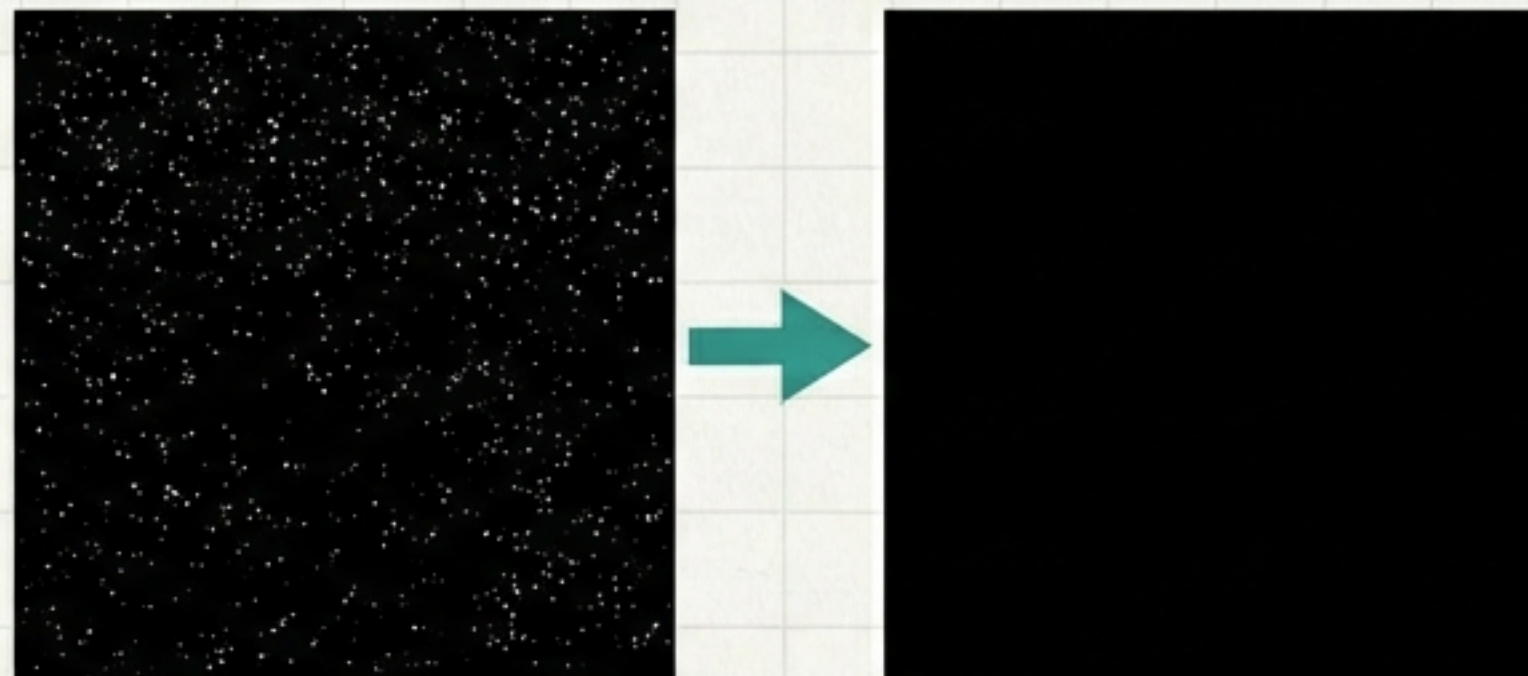
# Bộ lọc Min & Max

## Bộ lọc Max



**Lọc Max:** Chọn điểm sáng nhất  
→ Xóa điểm đen (Pepper).

## Bộ lọc Min



**Lọc Min:** Chọn điểm tối nhất  
→ Xóa điểm trắng (Salt).

Đây là cơ sở cho các phép toán Hình thái học (Morphology).



# Tổng kết Tiết 5

Ảnh Thực tế (Nhiều)



Median Filter



Ảnh Sạch



Sobel/Laplace



Đường Biên (Cấu trúc)



## Key Takeaways:

1. **Phát hiện biên:** Dựa trên sự biến thiên cường độ (**Gradient**).
2. **Công cụ:** Sobel (có hướng), Laplacian (vô hướng).
3. **Khôi phục:** **Median Filter** là giải pháp tối ưu cho nhiễu muối tiêu.



# Sự kết hợp hoàn hảo - Thuật toán Canny



Những gì đã học hôm nay là nguyên liệu cho thuật toán Canny:

- Gaussian Blur (Khử nhiễu)
- Sobel Operator (Tìm Gradient)
- Non-max Suppression (Làm mảnh)
- Hysteresis Thresholding (Lọc biên)



# Hướng dẫn Thực hành Lab 2.2

## Nhiệm vụ:

1. **Custom Kernels:** Tự viết kernel và dùng `cv2.filter2D`.
2. **Pencil Sketch:** Kết hợp Sobel + Invert color.
3. **Denoise Challenge:** Khôi phục ảnh nhiễu bằng `cv2.medianBlur`.

```
# Ví dụ Median Blur  
restored_img = cv2.medianBlur(noisy_img, 5)
```

```
1 import python
2
3
4 import cv2
5 import cv2 as ID
6
7 noisy__img = noisy__img
8 noisy__img = nota.a_aK
9
10
11
12
13
14
15
16
17 def noisy_img, cv2.medianBlur(noisy_img):
18
19     restored_img = cv2.filter2D(
```